



< RÉSOLVEZ DES PROBLÈMES EN UTILISANT DES ALGORITHMES EN PYTHON >

CONTEXTE DU PROJET

- Créer un algorithme qui extrait la meilleure combinaison d'achat d'actions en bourse d'après un fichier donné et en respectant plusieurs contraintes.
- Développer tout d'abord une solution « brute force », puis une solution optimisée
- Comparer l'efficacité et les performances des 2 solutions

CONTRAINTES :

- Chaque action ne peut être achetée qu'une seule fois.
- Nous ne pouvons pas acheter une fraction d'action
- Nous pouvons dépenser au maximum 500 euros par client

COMPLEXITÉ SPATIALE ET TEMPORELLE BRUTEFORCE

- La complexité spatiale de l'algorithme brute force est principalement liée à la fonction qui génère une liste de toutes les combinaisons possibles. Au plus, le nombre de combinaisons possibles est de 2^n où n représente le nombre d'actions.

La complexité spatiale de cet algorithme est de $O(2^n)$

- Cet algorithme utilise 2 boucles imbriquées pour générer toutes les combinaisons possibles. La première va réaliser 2^{n-1} itérations, la 2^e quant à elle va itérer n fois.

La complexité temporelle de cet algorithme est donc de $O(n \cdot 2^{n-1})$

COMPLEXITÉ SPATIALE ET TEMPORELLE

VERSION OPTIMISÉE

Complexité temporelle:

La fonction utilise deux boucles imbriquées, une pour parcourir les actions et une pour parcourir les valeurs du budget.

Le nombre d'itérations de la boucle externe est « number_of_shares », qui est le nombre d'actions (que l'on nommera « n »).

Le nombre d'itérations de la boucle interne est au maximum budget, car on parcourt toutes les valeurs du budget jusqu'à ce qu'on trouve une solution.

La complexité temporelle est donc $O(n \cdot \text{budget})$

Complexité spatiale:

La fonction utilise deux tableaux à deux dimensions, « results » et « chosen_shares ».

La taille de ces tableaux est de « number_of_shares » * « budget ».

La complexité spatiale est donc $O(n \cdot \text{budget})$

PSEUDOCODE DE L'ALGORITHME OPTIMISÉ

Entrée :

shares_list : Une liste d'actions, chaque action étant représentée par un dictionnaire avec des clés "name", "price" et "earnings".

budget: Le budget maximal pour investir.

Sortie:

best_combination: Une liste contenant les noms des actions qui composent la meilleure combinaison d'investissement pour le budget donné.

Algorithme

On Initialiser une liste à 2 dimensions pour stocker les résultats intermédiaires

Il y aura une rangée pour chaque nombre possible d'actions / budget, y compris 0 action jusqu'à la valeur maximale

Initialiser une liste à 2 dimensions pour stocker les actions sélectionnées pour chaque budget

Pour chaque action

Obtenir le prix et le profit de l'action

 Pour chaque valeur du budget

 On vérifie que le prix de l'action courante est inférieur au budget courant

 Si oui, on vérifie si son profit est plus intéressant

 Si c'est le cas on met à jour le budget en y soustrayant le prix de l'action

 On ajoute l'action à la liste d'actions sélectionnées

 sinon:

 Si l'action courante n'est pas assez rentable:

 on met à jour les valeurs

 sinon:

 Si l'action courante n'entre pas dans le budget

 on met à jour les valeurs

On obtient la combinaison la plus rentable qui tient compte du budget imposé

RÉSULTAT AVEC LA MÉTHODE BRUTEFORCE (DATA1)

- La meilleure combinaison d'achat est la suivante :
['Action-4', 'Action-5', 'Action-6', 'Action-8', 'Action-10', 'Action-11',
'Action-13', 'Action-18', 'Action-19', 'Action-20']
- Cette combinaison a un coût total de 498.0€ et rapportera 99.08€

RÉSULTATS AVEC LA MÉTHODE OPTIMISÉE (DATASET1)

- Choix et gains de Sienna pour le Dataset1 :
Share-GRUT
- Coût total : 498.76€
- Gains : 196,61€

- La meilleure combinaison d'achat, avec notre algorithme, pour le Dataset1 est la suivante :
['Share-IFCP', 'Share-EMOV', 'Share-KZBL', 'Share-LRBZ', 'Share-XJMO', 'Share-GIAJ', 'Share-QQTU', 'Share-SKKC', 'Share-ZSDE', 'Share-LGWG', 'Share-WPLI', 'Share-QLMK', 'Share-FKJW', 'Share-GTQK', 'Share-USSR', 'Share-MTLR', 'Share-LPDM', 'Share-UEZB', 'Share-NHWA', 'Share-GHIZ', 'Share-KMTG']
- Coût total : 499.95€
- Gains : 198.54€

RÉSULTATS AVEC LA MÉTHODE OPTIMISÉE (DATASET2)

Choix de Sienna pour le Dataset2 :

['Share-ECAQ', 'Share-IXC', 'Share-FWBE', 'Share-ZOFA', 'Share-PLLK', 'Share-YFVZ', 'Share-ANFX', 'Share-PATS', 'Share-NDKR', 'Share-ALIY', 'Share-JWGF', 'Share-JGTW', 'Share-FAPS', 'Share-VCAX', 'Share-LFXB', 'Share-DWSK', 'Share-XQII', 'Share-ROOM']

- **Coût total : 489,24€**
- **Gains : 193,78€**

La meilleure combinaison d'achat, avec notre algorithme ,pour le Dataset2 est la suivante :

['Share-ROOM', 'Share-XQII', 'Share-DWSK', 'Share-LFXB', 'Share-VCAX', 'Share-FAPS', 'Share-JGTW', 'Share-JWGF', 'Share-ALIY', 'Share-NDKR', 'Share-SCWM', 'Share-PATS', 'Share-ANFX', 'Share-YFVZ', 'Share-LXZU', 'Share-PLLK', 'Share-ZOFA', 'Share-FWBE', 'Share-IXCI', 'Share-ECAQ']

- **Coût total : 499,92€**
- **Gains : 197,95€**

PERFORMANCES



Algorithme Bruteforce

Data 1 (20 lignes) :

- Tps exécution : 4,36 secondes
- RAM utilisée : 218,54 Mio

Algorithme Optimisé

Data 1 (20 lignes) :

- Tps exécution : 0,75 secondes
- RAM utilisée : 67,77 Mio

Algorithme Optimisé

Dataset 1 :

- Tps exécution : 36,63 secondes
- RAM utilisée : 67,83 Mio

Dataset 2 :

- Tps exécution : 18,71 secondes
- RAM utilisée : 67,78 Mio

CONCLUSION

- Notre algorithme est plus performant que celui de Sienna pour le dataset1, il l'est moins pour la dataset2 mais nous sommes plus proche du budget de 500€ indiqué dans la consigne de départ.
- D'autre part nous n'avons aucun résultat concernant les performances de l'algorithme de Sienna. Combien de temps met-il à s'exécuter? Combien de RAM utilise-t-il? Ces informations seraient utiles pour définir l'algorithme qui, au final, sera le plus performant.